K.R. MANGALAM UNIVERSITY

School of Engineering & Technology

ENSP465 — New Age Programming Languages Lab

CAPSTONE PROJECT — PHASE I

PROJECT PROPOSAL & SYSTEM DESIGN

PROJECT TITLE

LIBRARY MANAGEMENT SYSTEM USING GO

Submitted By:

Name : Kartik

Course: B.sc Computer Science

Roll NO: 2401720011

Name: Himanshu

Course: B.sc Cyber Security

Roll NO. 2401830003

Submitted To:

Kumawat Hemraj

Assistant Professor

(SOET)

Project Summary

This project is a Library Management System for a small college library. It will be made using Go, with SQLite used to save the records. The system will help a librarian manage books, members, and book issue and return details.

2. Problem Statement

Problem Description

A library keeps records of books, members, and borrowed books. If records are maintained in registers, it can take time to find information. Mistakes can also happen while checking available books or noting return dates.

Real-World Context

The system is intended for a small library managed by one librarian on one computer. It will provide basic options to add books and members, issue and return books, and view overdue books.

3. Proposed Solution and Language Justification

Project Overview

The program will have a simple text menu for book records, member records, issue/return operations, searches, and overdue fines. SQLite will store the information so it is available the next time the program is run.

Chosen Language: Go

I chose Go because is suitable for building a small command-line program.

Technical Justification

Go is suitable for this project for three main reasons: structs can store book and member details, error handling helps report invalid operations, and packages such as time and database/sql help with dates and database access. The program can also be divided into simple functions for each task.

Planned Libraries and Tools

I plan to use fmt and bufio for menu input and output, time for dates, and database/sql with a SQLite driver to store records.

4. Objectives and Scope

Measurable Objectives

1. Add, view, update, and delete book records. 2. Register and search library members. 3. Issue and return books while checking availability. 4. Store due dates and calculate a simple overdue fine. 5. Save records in SQLite and display a basic list of available and overdue books.

In-Scope Features

• Text-based menu. • Add, view, edit, and remove books. • Add and search members. • Issue and return books. • Check available copies and due dates. • Calculate overdue fines. • Save data in SQLite. • Show simple book and overdue lists.

Out-of-Scope Features

• Website or mobile app. • Barcode scanner. • Email or SMS reminders. • Online fine payments. • Multiple branches or cloud access. • Advanced login and permissions.

Assumptions and Constraints

The first version is designed for one librarian operating the application locally. Each physical book copy is represented through a total-copy count and an available-copy count, while individual loans reference the book and member. The fine rate and loan duration will be configurable constants or simple settings. The system will not replace institutional policy; the librarian must confirm local borrowing rules.

5. System Architecture and Design

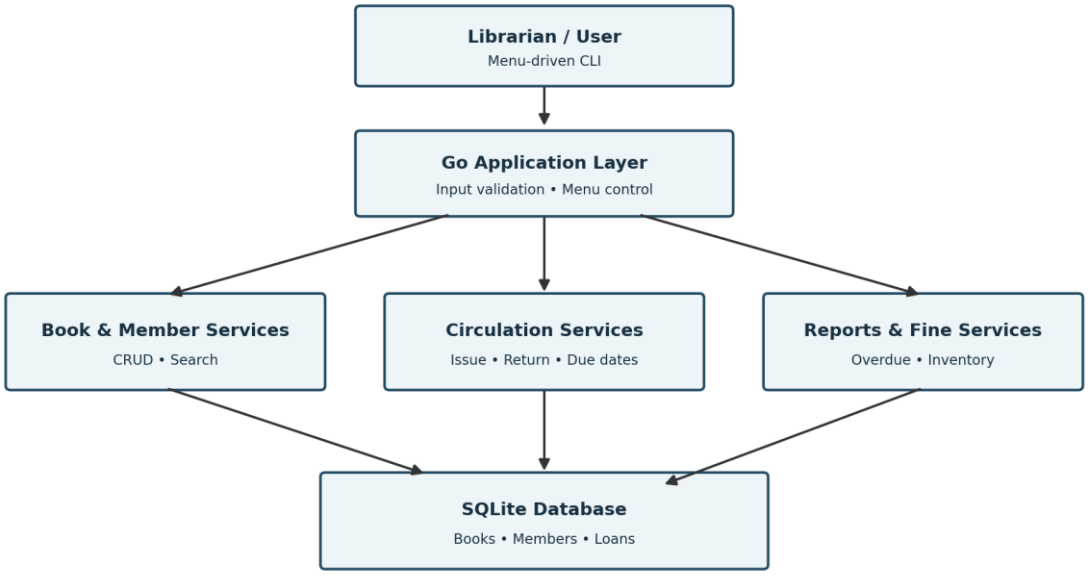
Architecture Overview

The system will have three main parts: a menu for the librarian, Go functions that process each choice, and an SQLite database that stores the records. Separate functions will handle books, members, issuing and returning, and reports.

Component Descriptions

1. Menu: takes the librarian's choice and displays messages. 2. Book functions: add, update, search, and show books. 3. Member functions: register and search members. 4. Issue/return functions: check availability, save borrowing details, and update book counts. 5. Database: stores book, member, and borrowing records.

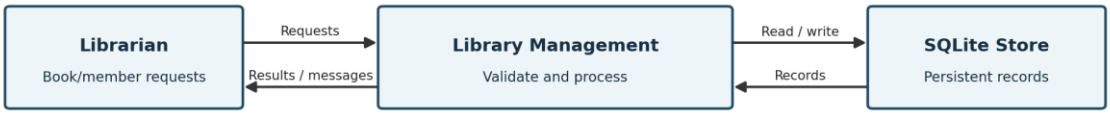
Figure 1. High-Level System Architecture



Data Flow Diagram

Figure 2 shows the basic data flow: the librarian enters a request, the program processes it and accesses the database, then displays the result.

Figure 2. Data Flow Diagram — Level 0



Inputs: book details, member details, issue/return requests

Outputs: confirmations, availability, due dates, fine and reports

Input–Process–Output (IPO)

Input: book and member details, plus issue or return choices. Process: check the entered details, confirm that a book is available, save the transaction, and calculate any overdue days. Output: a confirmation message, due date, book availability, or overdue list.

Key Data Structures

Go structs will be used for Book, Member, and IssueRecord. A book will contain its ID, title, author, and available copies. A member will contain an ID and name. An issue record will contain the book ID, member ID, issue date, due date, and return date.

Database Design

The database will have three tables: Books (book ID, title, author, and available copies), Members (member ID and name), and Issues (issue ID, book ID, member ID, issue date, due date, and return date). These tables connect each borrowing record to a book and a member.

Error Handling and Security Considerations

The program will check inputs and show a message when an entered ID is invalid or a book is unavailable. Since this is a student project with no advanced login system, it is intended as a local demonstration rather than a finished commercial product.

6. Work Plan and Timeline

Milestone Plan

The following is a tentative four-to-six milestone plan. Dates should be adjusted to the faculty's approved Phase II and Phase III schedule.

Deliverable: proposal, diagrams, objectives, and references.

basic Go project, database connection, tables, and menu. book and member options with search and saved records.

issue/return, due dates, fines, and basic reports.test common cases, fix errors, and demonstrate the program.source code, short user guide, test results, final report, and viva preparation.

Task Distribution

For an individual project, one student is responsible for requirements, design, implementation, testing, documentation, and presentation. If converted into a team project, responsibilities can be split between database/design, catalogue/member modules, circulation/fines, and testing/documentation.